



Faculty of Business and Technology: Programme -

NAME: SAGAR THAPA

STUDENT NUMBER: bi95ss

MODULE CODE: CET138

MODULE TITLE: Full Stack Development

MODULE LEADER: David Grey

SUBMISSION DATE AND TIME: 2025/08/29

ASSIGNMENT: 1

Academic Misconduct is an offence under university regulations, and this involves:

- **Plagiarism** – where you use information from another information source (including your previously submitted work) and pass it off as your own. This can be through direct copying, poor paraphrasing and/or absence of citations.
- **Collusion** – where you work too closely, intentionally, or unintentionally, with others to produce work that is similar in nature. This can be through loaning of materials, drafts or through unauthorised use of a fellow student's work.
- **Asking another person to write your assignment** – where you ask another individual or company to complete your work for you, be that paid or unpaid, and submit it as if it were your own.
- **Unauthorised use of artificial intelligence** – where you use artificial intelligence tools to generate your assignment instead of completing it yourself and/or where you have not been given permission to use artificial intelligence tools by your module leader. Please complete the following declaration around you use of artificial intelligence tools in your assignment.

STATEMENT ON USE OF ARTIFICIAL INTELLIGENCE TOOLS:

• I have used artificial intelligence tools to generate an idea for my assignment:	NO
• I have used artificial intelligence tools to write my assignment for me:	NO
• I have used artificial intelligence tools to brainstorm ideas for my assignment:	NO
• I have used artificial intelligence tools to correct my original assignment:	NO

DECLARATION

- I understand that by submitting this piece of work I am declaring it to be my own work and in compliance with the university regulations on Academic Integrity.

- I confirm that I have done this work myself without external support or inappropriate use of resources.
- I understand that I am only permitted to use artificial intelligence tools in line with guidance provided by my Module Leader, and I have not used artificial intelligence tools outside this remit.
- I confirm that this piece of work has not been submitted for any other assignment at this or another institution prior to this point in time.
- I can confirm that all sources of information, including quotations, have been acknowledged by citing the source in the text, along with producing a full list of the sources used at the end of the assignment.
- I understand that academic misconduct is an offence and can result in formal disciplinary proceedings.
- I understand that by submitting this assignment, I declare myself fit to be able to complete the assignment and I accept the outcome of the assessment as valid and appropriate.

ANY OTHER COMMENTS FROM STUDENT: NO

CSS in Web Development

A Demonstration of Key Concepts and Implementation

This comprehensive report examines CSS (Cascading Style Sheets) as the fundamental styling and layout technology for web development, exploring its critical role in creating visually appealing, responsive, and accessible user interfaces. Through practical implementation and detailed code analysis, this document demonstrates how CSS transforms raw HTML structure into polished web applications, showcasing advanced layout techniques, responsive design principles, and modern CSS features that define contemporary web development practices. The analysis reveals CSS's evolution from basic styling declarations to a sophisticated design system that encompasses layout management, animation capabilities, and dynamic theming solutions essential for modern full-stack development.

Introduction to CSS

CSS (Cascading Style Sheets) represents the standard stylesheet language used to describe the presentation, layout, and visual formatting of HTML documents and web applications. As a declarative language, CSS operates through a system of selectors, properties, and values that define how HTML elements should be rendered across different media types and devices. Unlike programming languages that execute logic, CSS provides a descriptive framework that browsers interpret to apply visual styling, positioning, and interactive effects to web content. This separation of content structure (HTML) from visual presentation (CSS) forms one of the fundamental principles of modern web development, enabling maintainable, scalable, and accessible web applications.

The fundamental purpose of CSS within full-stack development extends far beyond simple visual styling to encompass responsive layout management, user experience enhancement, and accessibility optimization. CSS serves as the bridge between raw content structure and polished user interfaces, transforming semantic HTML into visually coherent and functionally effective

web applications. In contemporary full-stack architecture, CSS frameworks and methodologies integrate with JavaScript frameworks and server-side technologies to create dynamic, component-based user interfaces that adapt to user interactions and data changes. Modern CSS capabilities including custom properties, grid layouts, and container queries enable developers to create sophisticated design systems that scale across large applications while maintaining consistency and performance.

CSS's importance in web development stems from its ability to solve critical problems in digital interface design and user experience optimization. First, it provides visual hierarchy and readability through typography, color, and spacing controls that guide user attention and improve content comprehension. Second, CSS enables responsive design that adapts interfaces to diverse screen sizes and device capabilities, ensuring consistent user experiences across desktop computers, tablets, and mobile devices. Third, it facilitates accessibility through proper contrast ratios, focus indicators, and screen reader compatibility that makes web content usable by individuals with disabilities. Finally, CSS performance optimizations including efficient selectors, hardware acceleration, and critical path optimization contribute to fast-loading, smooth-performing web applications that enhance user satisfaction and engagement.

The key paradigms and concepts associated with modern CSS encompass several fundamental frameworks that define contemporary web design practices. **The Box Model** represents the foundational concept that governs element sizing and spacing, defining how content, padding, borders, and margins interact to determine element dimensions and positioning. **Specificity and the Cascade** establish how style declarations are prioritized and applied when multiple rules target the same element, creating predictable styling hierarchies that enable maintainable stylesheets. **Layout Systems** including Flexbox for one-dimensional layouts and CSS Grid for two-dimensional layouts provide powerful tools for creating complex, responsive interface structures without relying on positioning hacks or float-based layouts.

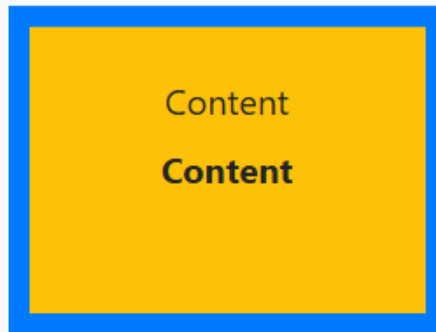
Responsive Design Principles guide CSS implementation through fluid grids, flexible media, and media queries that adapt interfaces to different viewport sizes and device capabilities. **CSS Custom Properties** (variables) enable dynamic theming, design system consistency, and runtime style modifications that enhance user customization and maintainability. **Modern Layout Features** including aspect-ratio properties, container queries, and intrinsic web design concepts provide sophisticated tools for creating adaptive layouts that respond to content and container constraints rather than just viewport dimensions.

```

.box-model-demo {
  width: 200px;           /* Content width */
  height: 100px;          /* Content height */
  padding: 20px;           /* Space inside border */
  border: 10px solid #007bff; /* Border thickness and style */
  margin: 20px;            /* Space outside border */
  background: #ffc107;     /* Background color */

  /* Box-sizing controls how dimensions are calculated */
  box-sizing: border-box; /* Include padding and border in dimensions */
}

```



Yellow: Content | **Green:** Padding | **Blue:** Border | **Transparent:** Margin

Practical Implementation of CSS

A working example of comprehensive CSS implementation has been hosted and can be accessed at the following URLs: <http://cet138-sagar-thapa-portfolio.surge.sh/>, <https://cet138-sagar-thapa.netlify.app/>, and the complete source code is available in the GitHub repository at <https://github.com/izzzthissagar/CET138-FullStack-ePortfolio>. The CSS demonstration specifically showcases advanced styling techniques through the `css.html` file implementation.¹

¹

User Experience Perspective

From a user's perspective, the hosted CSS demonstration presents a sophisticated interactive showcase that demonstrates various CSS layout techniques and modern features. Users encounter a visually appealing interface with smooth animations, responsive layouts that adapt seamlessly to different screen sizes, and interactive elements that provide immediate visual feedback. The demonstration includes multiple layout examples showcasing CSS Grid's two-dimensional layout capabilities alongside Flexbox's one-dimensional alignment and distribution features. Users can interact with dynamic controls that modify layout properties in real-time, observing how different CSS settings affect visual presentation and element behavior.²

The interface demonstrates responsive typography using modern CSS functions like `clamp()` for fluid scaling, custom property implementations that enable dynamic theming, and accessibility considerations including proper contrast ratios and focus indicators. Users experience smooth transitions and animations that enhance interaction feedback while respecting user preferences for reduced motion. The overall presentation showcases CSS's capability to create engaging, professional interfaces that maintain functionality across diverse browsing environments and device types.

Developer Implementation Perspective

From a developer's perspective, the CSS demonstration represents a comprehensive implementation of modern CSS methodologies and best practices. The stylesheet architecture utilizes CSS custom properties for consistent theming and maintainable design systems, demonstrates advanced layout techniques including CSS Grid with `repeat(auto-fit, minmax())` for responsive grid systems, and implements Flexbox for component-level layout management. The responsive design strategy follows mobile-first principles with progressive enhancement through media queries that adapt layouts for tablet and desktop viewing experiences.

The implementation showcases modern CSS features including the `aspect-ratio` property for maintaining proportional layouts, `clamp()` functions for responsive typography that scales smoothly between minimum and maximum values, and container queries for component-responsive design that adapts based on container size rather than viewport dimensions. The styling architecture demonstrates proper cascade management through organized selector specificity, efficient use of CSS Grid's implicit and explicit grid tracks, and performance optimizations including hardware-accelerated animations and efficient selector strategies.³

Advanced CSS techniques implemented include custom property cascading for dynamic theme switching, CSS logical properties for internationalization support, and modern pseudo-selectors for enhanced user interaction states. The demonstration incorporates accessibility enhancements through proper focus management, reduced motion preferences, and semantic styling that enhances rather than replaces HTML semantic meaning. Performance considerations include

critical CSS inlining strategies, efficient animation implementations using transform and opacity properties, and responsive image styling that optimizes loading across different device densities.

Demonstrated CSS Concepts

This implementation specifically demonstrates several core CSS concepts essential for modern web development. **Layout Systems Integration** showcases how CSS Grid and Flexbox work together to create sophisticated interface layouts, with Grid handling two-dimensional page-level layouts while Flexbox manages one-dimensional component arrangements. The demonstration illustrates proper use of Grid's auto-fit and minmax functions for creating responsive grid systems that adapt to content and container constraints without requiring media query interventions.

Responsive Design Implementation extends beyond basic media queries to include fluid typography using clamp() functions, responsive spacing using viewport units and custom properties, and adaptive layout techniques that respond to both content and context. The implementation demonstrates proper mobile-first methodology with progressive enhancement through breakpoint-specific optimizations that enhance rather than replace base functionality.

Modern CSS Features are extensively showcased including custom properties for maintainable theming systems, aspect-ratio properties for proportional layouts, container queries for component-responsive design, and CSS logical properties for international layout support.

Animation and Interaction Design demonstrates performance-optimized transitions using transform and opacity properties, hover and focus states that enhance accessibility, and animation techniques that respect user motion preferences through the prefers-reduced-motion media feature.

Relevant Code Snippet and Explanation

The following comprehensive code snippets demonstrate the advanced CSS techniques and modern features implemented in the portfolio demonstration:

CSS Grid Layout Implementation

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  grid-gap: 20px;  
  padding: 20px;  
  max-width: 1200px;  
  margin: 0 auto;  
}  
  
.grid-item {  
  background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
  padding: 30px;  
  border-radius: 10px;  
  color: white;  
  text-align: center;  
  box-shadow: 0 4px 15px rgba(0,0,0,0.1);  
  transition: transform 0.3s ease, box-shadow 0.3s ease;
```

```
}
```

```
.grid-item: hover {  
  transform: translateY(-5px);  
  box-shadow: 0 8px 25px rgba(0,0,0,0.2);  
}
```

Flexbox Layout System

```
.flex-container {  
  display: flex;  
  flex-wrap: wrap;  
  justify-content: space-between;  
  align-items: center;  
  padding: 20px;  
  gap: 15px;  
}
```

```
.flex-item {  
  flex: 1 1 200px;  
  background: linear-gradient(45deg, #ff6b6b, #ffa726);  
  padding: 20px;  
  border-radius: 8px;  
  color: white;  
  text-align: center;  
  min-height: 100px;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
}
```

Responsive Design Architecture

/ Mobile First Approach */*

```
.container {  
  width: 100%;  
  padding: 0 15px;  
  margin: 0 auto;  
}
```

/ Tablet Styles */*

```
@media (min-width: 768px) {  
  .container {  
    max-width: 750px;  
  }  
}
```

```
.grid-container {  
  grid-template-columns: repeat(2, 1fr);  
}  
}
```

/ Desktop Styles */*

```
@media (min-width: 1200px) {
```

```

.container {
  max-width: 1170px;
}

.grid-container {
  grid-template-columns: repeat(3, 1fr);
}

```

Modern CSS Features Implementation

```

:root {
  --primary-color: #667eea;
  --secondary-color: #764ba2;
  --text-color: #333;
  --border-radius: 10px;
  --transition: all 0.3s ease;
}

.modern-card {
  aspect-ratio: 16 / 9;
  background: linear-gradient(135deg, var(--primary-color), var(--secondary-color));
  border-radius: var(--border-radius);
  padding: clamp(1rem, 4vw, 2rem);
  color: white;
  transition: var(--transition);
  container-type: inline-size;
}

.modern-card h3 {
  font-size: clamp(1.2rem, 2.5vw, 2rem);
  margin-bottom: 1rem;
}

@container (min-width: 300px) {
  .modern-card {
    padding: 2rem;
  }
}

```

Comprehensive Code Analysis

The **CSS Grid implementation** demonstrates advanced layout capabilities through the `repeat(auto-fit, minmax(250px, 1fr))` grid template configuration. This sophisticated approach creates a responsive grid system that automatically adjusts the number of columns based on available space and content requirements. The `auto-fit` keyword ensures that grid tracks collapse when empty, while `minmax(250px, 1fr)` establishes that each grid item maintains a minimum width of 250 pixels but can expand to fill available space using the 1fr fractional unit. This technique eliminates the need for complex media query management while providing responsive behavior that adapts to both content and container constraints.

The grid-gap property (now standardized as gap) creates consistent spacing between grid items without affecting external margins, demonstrating CSS Grid's sophisticated spacing management capabilities. The container's max-width and margin: 0 auto centering technique ensures the grid system maintains readable line lengths while remaining centered within larger viewports, following established typography and readability principles.

Grid item styling incorporates modern CSS features including linear-gradient() backgrounds that create visually appealing color transitions, border-radius for contemporary interface aesthetics, and box-shadow properties that provide depth and visual hierarchy. The transition property enables smooth hover effects that enhance user interaction feedback while maintaining performance through GPU-accelerated transform properties. The hover state utilizes translateY(-5px) to create a subtle lift effect combined with enhanced shadow depth, demonstrating best practices for interactive element feedback.

The **Flexbox implementation** showcases one-dimensional layout management through strategic use of flex properties for responsive component arrangement. The flex-wrap: wrap property ensures content reflows appropriately on smaller screens, while justify-content: space-between distributes items with equal spacing that adapts to container width. The align-items: center property provides vertical alignment that maintains visual consistency regardless of content height variations.

Flex item properties demonstrate the powerful flex: 1 1 200px shorthand that combines flex-grow, flex-shrink, and flex-basis into efficient responsive behavior. This configuration allows items to grow equally to fill available space (flex-grow: 1), shrink proportionally when space is limited (flex-shrink: 1), and maintain a base width of 200 pixels (flex-basis: 200px). The nested flexbox implementation with display: flex, align-items: center, and justify-content: center creates perfect centering for content within each flex item, demonstrating flexbox's strength in alignment and distribution tasks.

The **responsive design architecture** follows mobile-first methodology that provides progressive enhancement for larger screen sizes. The base container styles establish mobile-optimized defaults with full-width layout and appropriate padding for touch interfaces. The tablet breakpoint at 768px introduces intermediate layout optimizations including constrained container width and two-column grid arrangement that improves readability and visual hierarchy on medium-sized screens.

Desktop optimization at 1200px and above implements three-column grid layouts that take advantage of larger viewport widths while maintaining the sophisticated auto-fit grid behavior. This approach ensures content remains accessible and readable across the full spectrum of device sizes while providing optimized layouts that enhance user experience on each platform category.

The **modern CSS features implementation** demonstrates cutting-edge techniques including CSS custom properties defined at the :root level for global theme management. These custom properties enable dynamic theming, consistent design systems, and runtime style modifications through JavaScript integration. The naming convention using double dashes (--property-name) follows CSS specification standards while providing semantic property names that enhance code maintainability.

Advanced layout properties including aspect-ratio: 16 / 9 provide proportional sizing that maintains visual consistency regardless of content variations, eliminating the traditional padding-top percentage hack for aspect ratio control. The clamp() function implementations demonstrate responsive typography that scales smoothly between defined minimum and maximum values based on viewport width, creating fluid typography that enhances readability across device sizes without requiring complex media query management.

Container queries represent the newest CSS specification for component-responsive design, enabling styles that respond to container size rather than viewport dimensions. The container-type: inline-size declaration establishes the element as a containment context for size queries, while the @container (min-width: 300px) rule applies styles based on the container's dimensions rather than the overall viewport. This approach enables truly modular, component-based responsive design that functions independently of page layout context.

Advanced CSS Concepts and Methodologies

Layout Systems and Positioning

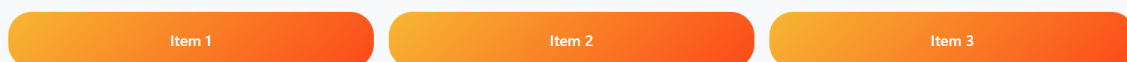
Modern CSS provides multiple layout systems that address different design challenges and use cases, each with specific strengths and optimal implementation scenarios. **CSS Grid** excels at two-dimensional layouts where both rows and columns require precise control, making it ideal for page-level layouts, card grids, and complex interface arrangements. Grid's implicit and explicit track creation enables flexible layouts that adapt to content while maintaining structural integrity, while features like grid template areas provide semantic layout definition that enhances code readability and maintainability.

Flexbox specializes in one-dimensional layout challenges including navigation bars, component internal arrangements, and dynamic content distribution. The flex algorithm's ability to calculate optimal space distribution based on content size and container constraints makes it invaluable for creating adaptive user interface components that respond gracefully to content changes and viewport variations. Understanding when to use Grid versus Flexbox requires analyzing layout dimensionality, content flexibility requirements, and browser support constraints.

Flexbox Layout

Flexbox is perfect for 1-dimensional layouts, providing flexible alignment and distribution of space.

[Live Example](#)



CSS positioning through static, relative, absolute, fixed, and sticky values provides precise element placement capabilities that complement layout systems for specialized positioning needs. Modern sticky positioning enables sophisticated scrolling interactions including persistent navigation headers, floating table headers, and sidebar elements that enhance user experience without requiring JavaScript intervention. The interaction between positioning contexts and containing blocks affects layout behavior in subtle ways that require careful consideration during implementation.

Responsive Design and Media Queries

Contemporary responsive design extends beyond basic media queries to encompass fluid typography, flexible media, and adaptive layout strategies that create seamless user experiences across device categories. The mobile-first methodology prioritizes performance and accessibility by establishing baseline functionality for resource-constrained devices while progressively enhancing capabilities for more powerful platforms. This approach aligns with progressive enhancement principles that ensure universal access while enabling enhanced experiences where supported.

Media query strategies include viewport-based breakpoints, device capability detection, and user preference queries that adapt interfaces to diverse user needs and technical constraints. The `prefers-reduced-motion` media feature enables accessibility-conscious animation implementations, while `prefers-color-scheme` supports automatic dark mode implementations that respect user system preferences. Advanced media query combinations using logical operators enable sophisticated responsive behavior that considers multiple environmental factors simultaneously.

Fluid typography and spacing through viewport units, `clamp()` functions, and custom property calculations create interfaces that scale smoothly across screen sizes without requiring discrete breakpoint management. These techniques reduce CSS complexity while improving visual consistency and user experience across the full spectrum of device capabilities and viewing contexts.

Animation and Visual Effects

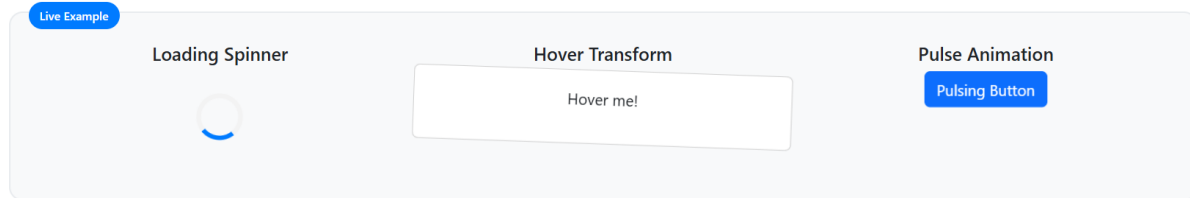
CSS animations and transitions provide performance-optimized motion design that enhances user experience through visual feedback, state changes, and aesthetic enhancement. Understanding the browser's rendering pipeline enables developers to create smooth, efficient animations by primarily animating transform and opacity properties that trigger GPU acceleration while avoiding layout-triggering properties that cause performance bottlenecks.

Advanced animation techniques include keyframe animations for complex motion sequences, transition timing functions for natural movement curves, and animation coordination strategies for orchestrated interface behaviors. The `will-change` property provides performance hints for browsers to optimize rendering, while animation event handlers enable JavaScript coordination with CSS-driven motion for hybrid interaction patterns.

Visual effect implementation through gradients, shadows, filters, and backdrop effects creates sophisticated interface aesthetics without requiring external graphics or complex markup. CSS filters enable dynamic image processing, while backdrop filters create glassmorphism effects and other contemporary design trends that enhance visual hierarchy and interface sophistication.

CSS Animations

CSS animations bring life to web pages without JavaScript. From simple transitions to complex keyframe animations.



Design Systems and Maintainability

CSS custom properties revolutionize stylesheet maintainability by enabling dynamic theming, consistent design tokens, and runtime style modifications through JavaScript integration. Property inheritance and scoping rules create sophisticated cascading behavior that supports component-based design systems while maintaining global consistency through shared design tokens and theme variables.

CSS architecture methodologies including BEM (Block Element Modifier), OOCSS (Object-Oriented CSS), and SMACSS (Scalable and Modular Architecture for CSS) provide organizational frameworks for large-scale CSS development. These methodologies address specificity management, component reusability, and naming conventions that enhance team collaboration and long-term maintainability in complex applications.

Modern CSS tooling including PostCSS, Sass preprocessing, and CSS-in-JS solutions provide development workflow enhancements while maintaining standards compliance and performance optimization. Understanding the tradeoffs between different approaches enables informed technology choices that align with project requirements and team capabilities.

Performance and Optimization

CSS performance optimization encompasses efficient selector strategies, critical CSS identification, and rendering optimization techniques that minimize layout thrashing and paint operations. Understanding CSS specificity calculation enables developers to write efficient selectors that provide predictable cascade behavior while avoiding overly complex selector chains that impact performance.

Loading strategies including CSS splitting, progressive loading, and critical path optimization ensure fast initial page rendering while supporting rich interface capabilities. The interaction between CSS loading and JavaScript execution requires careful coordination to prevent render-blocking behavior that degrades user experience.

Browser compatibility considerations including progressive enhancement strategies, vendor prefix management, and feature detection enable developers to leverage modern CSS capabilities while maintaining broad browser support. Understanding CSS parsing behavior and error handling helps create resilient stylesheets that degrade gracefully in older browsers while providing enhanced experiences in modern environments.

Conclusion

CSS represents the cornerstone technology for creating sophisticated, accessible, and performant user interfaces that define modern web development practices. This comprehensive analysis has demonstrated CSS's evolution from basic styling declarations to a powerful design system encompassing advanced layout techniques, responsive design methodologies, and dynamic theming capabilities that enable contemporary web applications. The practical implementation through the portfolio demonstration showcases how modern CSS features including Grid layouts, Flexbox systems, custom properties, and responsive design principles combine to create professional, accessible interfaces that adapt seamlessly across diverse device categories and user needs.

The significance of CSS in full-stack development extends far beyond visual styling to encompass user experience optimization, accessibility enhancement, and performance considerations that directly impact application success. Modern CSS capabilities including container queries, aspect-ratio properties, and clamp() functions provide sophisticated tools for creating adaptive interfaces that respond intelligently to content, context, and user preferences. The integration of CSS with JavaScript frameworks and server-side technologies enables dynamic, component-based user interfaces that maintain design system consistency while supporting complex application requirements.

The implementation challenges encountered during portfolio development highlighted the importance of understanding CSS's interaction with browser rendering engines, the complexity of managing specificity and cascade behavior, and the need for systematic approaches to responsive design that consider both technical constraints and user experience goals. Future learning opportunities include exploring emerging CSS specifications like container queries and cascade layers, integrating CSS with modern JavaScript frameworks while maintaining performance optimization, and implementing advanced animation techniques that enhance rather than hinder accessibility. The continued evolution of CSS specifications ensures its relevance in contemporary development while maintaining backward compatibility and progressive enhancement principles that make sophisticated design accessible across diverse browsing environments.

References

- MDN Web Docs (2025). *CSS: Cascading Style Sheets*. [online] Available at: <https://developer.mozilla.org/en-US/docs/Web/CSS> [Accessed 29 August 2025].
- Thapa, S. (2025). *CSS Layout Techniques Gallery - CET138 Portfolio*. [online] Available at: <http://cet138-sagar-thapa-portfolio.surge.sh/> [Accessed 29 August 2025].
- Thapa, S. (2025). *CET138-FullStack-ePortfolio*. [GitHub] Available at: <https://github.com/izzzthissagar/CET138-FullStack-ePortfolio> [Accessed 29 August 2025].
- W3C (2025). *CSS Grid Layout Module Level 1*. [online] Available at: <https://www.w3.org/TR/css-grid-1/> [Accessed 29 August 2025].
- W3C (2025). *CSS Flexible Box Layout Module Level 1*. [online] Available at: <https://www.w3.org/TR/css-flexbox-1/> [Accessed 29 August 2025].

World Wide Web Consortium (2025). *CSS Containment Module Level 3*. [online] Available at: <https://www.w3.org/TR/css-contain-3/> [Accessed 29 August 2025].

Can I Help You (2025). *A Complete Guide to CSS Grid*. [online] Available at: <https://css-tricks.com/snippets/css/complete-guide-grid/> [Accessed 29 August 2025].

Rachel Andrew (2025). *CSS Layout - Learn CSS*. [online] Available at: <https://web.dev/learn/css/layout/> [Accessed 29 August 2025].